

SOFTWARE REQUIREMENTS SPECIFICATION

AI-Enabled Telehealth Platform for Rural Healthcare Accessibility

Version 1.0

Document Type: Software Requirements Specification (SRS)

Development Methodology: Agile Incremental

Date: May 2026

Intended Audience

Developers • Testers • Project Reviewers • Mentors • Hackathon Evaluators • Future Maintainers

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional requirements, non-functional requirements, architecture, system modules, constraints, and overall software behavior of the AI-Enabled Telehealth Platform for Rural Healthcare Accessibility.

This document serves as the official reference for:

- ▶ Developers
- ▶ Testers
- ▶ Reviewers & Stakeholders
- ▶ Hackathon Evaluators

1.2 Scope of the Project

The AI-Enabled Telehealth Platform is a cloud-based healthcare system designed to improve healthcare accessibility for rural and remote communities. The platform enables:

- ▶ Remote doctor-patient consultation
- ▶ Electronic Health Record (EHR) management
- ▶ Appointment scheduling and medical report access
- ▶ AI-assisted symptom prioritization
- ▶ Secure healthcare data management
- ▶ Real-time video consultation using WebRTC

The system aims to reduce dependency on physical hospital visits while enabling secure and scalable healthcare services.

1.3 Problem Statement

Rural healthcare systems face several major challenges:

- ▶ Lack of specialist doctors in rural areas
- ▶ Long-distance travel required for medical consultation
- ▶ Poor healthcare accessibility and limited infrastructure
- ▶ Paper-based, non-digitized medical record management
- ▶ Delayed emergency response mechanisms

These challenges cause patients to face:

- Delayed treatment and poor health outcomes
- Increased healthcare expenses
- Poor patient monitoring
- Overcrowding in urban hospitals

1.4 Proposed Solution

Core Concept: An AI-enabled telehealth platform that allows rural patients to consult doctors remotely through real-time video conferencing while maintaining centralized electronic healthcare records.

The system integrates:

- ▶ MERN stack web application
- ▶ Real-time WebRTC consultation system
- ▶ Python-based AI symptom prioritization module
- ▶ Cloud-based deployment architecture

The AI module categorizes patients into:

Priority Level	Description
Critical	Requires immediate medical attention – highest urgency
Moderate	Requires prompt attention within a short window
Normal	Standard consultation, non-urgent cases

1.5 Objectives

The major objectives of the system are:

- ▶ Improve healthcare accessibility for rural communities
- ▶ Reduce patient dependency on long-distance travel
- ▶ Enable secure, standards-compliant telemedicine services
- ▶ Maintain centralized, accessible healthcare records
- ▶ Assist doctors through intelligent AI-driven patient prioritization
- ▶ Provide scalable, reliable cloud-based healthcare infrastructure

1.6 Development Methodology

Methodology: Agile Incremental Development Model

Agile is selected for this project because:

- ▶ Rapid development cycles are required
- ▶ Independent modules can be developed and tested simultaneously
- ▶ Faster iteration and feedback loops are possible
- ▶ Continuous testing and improvement is natively supported
- ▶ Well-suited for hackathon-style execution timelines

2. Overall Description

2.1 Product Perspective

The system is a full-stack cloud-based healthcare platform developed using:

Component	Technology
Frontend	React.js
Backend	Node.js + Express.js
Database	MongoDB Atlas
AI Service	Python AI Module
Video Communication	WebRTC
Deployment	AWS Cloud

2.2 Product Functions

The system provides the following major functionalities:

Authentication Functions

- ▶ Patient registration and account creation
- ▶ Secure login with encrypted credentials
- ▶ JWT-based stateless authentication
- ▶ Role-based access control (RBAC)

Patient Functions

- ▶ Book and manage appointments
- ▶ Enter and submit symptoms for AI analysis
- ▶ Upload and access medical reports
- ▶ Join real-time video consultations
- ▶ View prescriptions and consultation history

Doctor Functions

- ▶ View and manage patient appointments
- ▶ Access full patient records and history
- ▶ Conduct live video consultations
- ▶ Generate and manage prescriptions
- ▶ Review AI-analyzed symptom priority

AI Functions

- ▶ Automated symptom analysis
- ▶ Patient criticality categorization (Critical / Moderate / Normal)
- ▶ Priority-level assignment for queue management

EHR & Telehealth Functions

- ▶ Digital health record storage and retrieval
- ▶ Prescription and consultation history management
- ▶ Real-time peer-to-peer video via WebRTC

2.3 User Classes and Characteristics

User Class	Primary Role	Key Capabilities
Patient	Healthcare consumer	Books appointments, submits symptoms, joins consultations
Doctor	Healthcare provider	Conducts consultations, manages records, writes prescriptions
Admin	System operator	Monitors operations, manages users and system health

2.4 Operating Environment

Component	Environment	Details
Frontend	Web Browser	Any modern browser (Chrome, Firefox, Edge, Safari)
Backend	Node.js Runtime	Node.js v18+ with Express.js framework
AI Module	Python Runtime	Python 3.10+ with ML libraries
Database	MongoDB Atlas	Cloud-hosted managed NoSQL service
Hosting	AWS Cloud	EC2/ECS with load balancing
Containerization	Docker	Dockerized microservices architecture

2.5 Design Constraints

- ▶ Limited hackathon development timeline
- ▶ Rapid development requirement with minimal technical debt
- ▶ Limited AI model training resources and compute budget
- ▶ Cloud deployment within free-tier / budget constraints

2.6 Assumptions and Dependencies

Assumptions:

- ▶ All users have stable internet connectivity

- ▶ Users operate modern, standards-compliant web browsers
- ▶ AWS cloud services and MongoDB Atlas remain available

External Dependencies:

- ▶ MongoDB Atlas (cloud database service)
- ▶ WebRTC APIs (browser-native)
- ▶ Python 3.10+ environment
- ▶ AWS Cloud infrastructure
- ▶ Docker containerization platform

3. System Context

3.1 Existing System (Pain Points)

Current rural healthcare systems are predominantly:

- ▶ Offline and paper-based – no digital record-keeping
- ▶ Non-centralized – patient data siloed across locations
- ▶ Remote-inaccessible – no telehealth capability
- ▶ Heavily dependent on in-person hospital visits

3.2 Proposed System (Improvements)

The proposed system provides:

- ▶ Cloud-based healthcare access from any location
- ▶ Online consultation eliminating travel dependency
- ▶ Centralized, digitized medical records
- ▶ AI-assisted healthcare prioritization
- ▶ Real-time secure communication via WebRTC

3.3 System Workflow



4. System Architecture

4.1 Architecture Style

Pattern: Modular Layered Architecture using the MERN stack with a separate AI microservice.

4.2 High-Level Architecture

Layer / Component	Description
Frontend Layer (React.js)	Client-side SPA served over HTTPS; communicates via REST APIs
Backend Layer (Node.js + Express.js)	Handles all business logic, routing, authentication, and API orchestration
MongoDB Atlas	Persistent cloud NoSQL database for healthcare records and user data
Python AI Module	Standalone microservice for symptom analysis and priority categorization
WebRTC Service	Browser-native real-time peer-to-peer video consultation
AWS Cloud + Docker	Containerized cloud deployment with scalability and reliability

5. Technology Stack

Layer	Technology	Purpose
Frontend	React.js	Interactive UI with component-based architecture
Backend	Node.js + Express.js	Scalable REST API server
Database	MongoDB Atlas	Cloud NoSQL storage with flexible schema
AI Module	Python	Symptom analysis and ML integration
Authentication	JWT + bcrypt	Stateless secure auth with hashed passwords
Video Communication	WebRTC	Browser-native real-time P2P video
Deployment	AWS Cloud	Reliable, scalable cloud hosting
Containerization	Docker	Consistent cross-environment deployment
Version Control	GitHub	Source code management and collaboration

6. Technology Selection Rationale

Technology	Why Selected
React.js	Component-based frontend, fast rendering, reusable UI components, full MERN compatibility
Node.js	Non-blocking I/O architecture, fast API handling, highly scalable backend support
Express.js	Lightweight framework, easy REST API development, extensive middleware ecosystem
MongoDB Atlas	Flexible document schema, native JSON support, cloud scalability, MERN ecosystem fit
Python AI Module	Superior AI/ML ecosystem, easy symptom analysis, future ML model integration
WebRTC	Real-time P2P communication, ultra-low latency, browser-native with no licensing dependency
JWT Authentication	Stateless token-based auth, secure API access, horizontal scalability
Docker	Consistent deployments across environments, simplified containerized cloud delivery
AWS Cloud	Reliable infrastructure, global availability, elastic scaling

7. Functional Requirements

7.1 Authentication Module

ID	Requirement	Priority
FR-1	System shall allow patient self-registration with email and password	High
FR-2	System shall allow secure role-based login	High
FR-3	System shall hash all passwords using bcrypt before storage	High
FR-4	System shall issue and validate JWT tokens for all API access	High
FR-5	System shall enforce role-based authorization (Patient / Doctor / Admin)	High

7.2 Patient Module

ID	Requirement	Priority
FR-6	Patient shall be able to book and cancel appointments	High
FR-7	Patient shall be able to upload medical reports (PDF/image)	High
FR-8	Patient shall be able to enter and update symptom descriptions	High
FR-9	Patient shall be able to view generated prescriptions	Medium
FR-10	Patient shall be able to access full consultation history	Medium

7.3 Doctor Module

ID	Requirement	Priority
FR-11	Doctor shall be able to view and manage appointment queue	High
FR-12	Doctor shall be able to access complete patient health records	High
FR-13	Doctor shall be able to generate and store prescriptions digitally	High
FR-14	Doctor shall be able to initiate and conduct video consultations	High

7.4 AI Module

ID	Requirement	Priority
FR-15	Python AI module shall receive and analyze submitted symptoms	High
FR-16	System shall categorize patient urgency into Critical / Moderate / Normal	High

ID	Requirement	Priority
FR-17	AI module shall return a priority level to the backend for queue ordering	High

7.5 WebRTC Consultation Module

ID	Requirement	Priority
FR-18	System shall support live real-time video consultation sessions	High
FR-19	System shall establish secure peer-to-peer communication via signaling	High
FR-20	System shall support browser-based consultation with no plugin required	High

7.6 EHR Module

ID	Requirement	Priority
FR-21	System shall securely store and retrieve patient healthcare records	High
FR-22	System shall store and associate prescriptions with consultation records	High
FR-23	System shall maintain complete timestamped consultation history	Medium
FR-24	System shall store and link uploaded patient reports to EHR	Medium

8. Non-Functional Requirements

8.1 Performance

- ▶ API response time shall be under 500ms for standard requests
- ▶ Database queries shall be optimized with appropriate indexing
- ▶ Video consultation shall maintain stable frame rates with low latency

8.2 Security

- ▶ All passwords shall be hashed using bcrypt (min cost factor 10)
- ▶ All client-server communication shall be over HTTPS/TLS
- ▶ All API endpoints shall require valid JWT authentication
- ▶ Role-based authorization shall protect sensitive data routes
- ▶ PHI (Protected Health Information) shall be encrypted at rest

8.3 Scalability

- ▶ System shall support growing user base without architecture changes
- ▶ Containerized deployment shall allow horizontal scaling
- ▶ Database shall support multiple hospital integrations
- ▶ AI module shall be expandable with advanced ML models

8.4 Reliability

- ▶ Cloud deployment shall target 99.9% uptime SLA
- ▶ Database operations shall be ACID-compliant where applicable
- ▶ System shall implement error handling and graceful degradation

8.5 Usability

- ▶ UI shall be responsive across desktop and mobile browsers
- ▶ Patient and doctor dashboards shall provide intuitive single-page navigation
- ▶ Key workflows (booking, consultation) shall complete in 3 clicks or fewer

9. AI Module Architecture

9.1 AI Workflow



9.2 AI Communication

Protocol: The Node.js backend communicates with the Python AI service through internal REST APIs. The AI service runs as an independent microservice container.

Priority Category	Condition Description
Critical	Symptoms indicate life-threatening or urgent medical condition
Moderate	Symptoms suggest a condition requiring prompt medical attention
Normal	Routine consultation; symptoms are non-urgent or mild

10. WebRTC Architecture

10.1 WebRTC Workflow



10.2 Why WebRTC Over Third-Party Services?

Advantage	Explanation
Real-Time Communication	Ultra-low latency peer-to-peer streaming for seamless consultation
Browser-Native Support	No external software or plugins required for patients or doctors
Reduced Server Dependency	P2P architecture minimizes server load and bandwidth costs
Open Technology	No licensing fees or external platform dependencies
Privacy	Direct encrypted P2P connection reduces data exposure risk

11. Database Requirements

11.1 Database Technology

Database: MongoDB Atlas – Cloud-hosted managed NoSQL service

MongoDB Atlas is selected for:

- ▶ Flexible schema support – ideal for evolving healthcare data structures
- ▶ Cloud scalability – managed horizontal scaling
- ▶ Native JSON document storage – aligns with Node.js data flow
- ▶ Built-in security and compliance features

11.2 Major Collections

Collection	Description	Key Fields
Users	Authentication records for all user roles	userId, email, passwordHash, role
Patients	Patient profile and demographic data	patientId, name, DOB, contact
Doctors	Doctor profiles and specializations	doctorId, name, specialty, availability
Appointments	Booking and scheduling data	appointmentId, patientId, doctorId, datetime, priority
Consultations	Session records and notes	consultationId, appointmentId, notes, duration
Prescriptions	Digital prescriptions	prescriptionId, consultationId, medications
Medical Reports	Uploaded patient documents	reportId, patientId, fileUrl, uploadDate
Symptom Analysis	AI module outputs	analysisId, patientId, symptoms, priority
Notifications	In-app alerts and reminders	notificationId, userId, message, status

12. Security Requirements

Healthcare applications require robust, multi-layered security. The platform implements the following controls:

Security Control	Implementation Details
Password Hashing	bcrypt with minimum cost factor 10; salted hashes stored
Authentication	JWT tokens with expiry; refresh token rotation
Transport Security	HTTPS/TLS 1.2+ for all client-server communication
API Authorization	Protected routes requiring valid JWT for every request
Role-Based Access	Strict RBAC; patients cannot access doctor-only endpoints
Data Encryption	PHI encrypted at rest in MongoDB Atlas
Audit Logging	All sensitive operations logged with timestamp and user ID

13. Future Scope

The platform is designed with extensibility in mind. Planned enhancements include:

- ▶ Advanced Machine Learning diagnostic assistance modules
- ▶ Native mobile applications (iOS and Android)
- ▶ Real-time in-app messaging and chat support
- ▶ Multi-hospital and multi-clinic integration
- ▶ Healthcare analytics dashboard for administrators
- ▶ IoT healthcare device integration (vitals monitoring)
- ▶ Offline-capable progressive web app (PWA) for low-connectivity areas
- ▶ Telemedicine billing and insurance claim processing

14. Conclusion

The AI-Enabled Telehealth Platform is a scalable and secure healthcare solution engineered to bridge the critical gap in rural healthcare accessibility. By combining modern web technologies with intelligent AI prioritization and real-time telemedicine capabilities, the platform delivers a comprehensive, patient-centric healthcare experience.

The system integrates:

- ▶ Full-stack MERN development for rapid, reliable web application delivery
- ▶ Python-based AI processing for intelligent patient triage and prioritization
- ▶ WebRTC real-time consultation to eliminate the need for in-person visits
- ▶ Containerized AWS cloud deployment for scalability and availability
- ▶ Centralized Electronic Health Record management for continuity of care

Vision: To make quality healthcare accessible to every rural community, regardless of geographic barriers – through technology, intelligence, and compassion.